

SLT Dynamic Catalog

<https://gemini.google.com/app/0d5920bb57febc74>

User prompt: Today my supervisor told me to build a website but the requirements are extremely vague and unclear. I'll try to tell it to you as I remember. Can you help me understand the scope and what we are building? Here I go. The website should have nothing at first. Then there are templates, categories and subcategories. Items are in key value pairs. For example : We have nothing at all. We have a template builder. We build a template as "vehicles". We can add categories to it as cars, vans, bikes and so on. Then we can add subcategories to cars like petrol, diesel, electric and so on. Then we can add items. But the thing is, he said to use mongo db but there will be no tables as vehicles, cars, vans, electric or something like that. Why? And he requested role based access to create templates only, add values only and read only. What the heck is he asking me to build?

Response: Take a breath—what your supervisor asked for sounds confusing when described verbally, but it is actually a very classic software pattern!

He is asking you to build a **Dynamic Catalog Engine** (or a mini **Headless CMS**, similar to how tools like Strapi, Contentful, or Airtable work under the hood).

Instead of building a hardcoded website for "Vehicles" or "Electronics," you are building a flexible system where **users define their own data structures on the fly**.

What are you actually building?

You are building a **blank slate system** where an admin can define what kind of data the website holds without needing a developer to rewrite code or change database schemas every time a new category is added.

Here is the functional flow:

1. **Admin (Template Creator):** Starts with an empty system. Creates a template called "Vehicles" and defines key-value fields (e.g., brand: String, engineCapacity: Number).
2. **Category Builder:** Sets up categories and subcategories (e.g., Vehicles → Cars → Petrol).
3. **Data Contributor:** Selects a category and fills out dynamic form fields based on the key-value structure defined in the template.
4. **Viewer:** Browses and reads the data.

Why are there no MongoDB collections for vehicles, cars, or petrol?

If you created a new MongoDB collection (or SQL table) every time someone added a new category like "Vehicles" or "Laptops", your database architecture would constantly break and become impossible to maintain.

Instead, you use a **Metadata-Driven Database Architecture** (a dynamic schema pattern). Your MongoDB database will only ever need **3 or 4 fixed collections** total.

The 3 Core Collections You Need

1. Templates Collection

Defines *what fields exist* for a type of object.

```
{
  "_id": "template_123",
  "name": "Vehicles",
  "fields": [
    { "key": "brand", "label": "Brand Name", "type": "string" },
    { "key": "year", "label": "Model Year", "type": "number" },
    { "key": "price", "label": "Price (USD)", "type": "number" }
  ]
}
```

2. Categories Collection

Defines the hierarchy using a parent reference (tree structure).

```
[
  { "_id": "cat_cars", "name": "Cars", "parentId": null, "templateId": "template_123" },
  { "_id": "cat_petrol", "name": "Petrol", "parentId": "cat_cars", "templateId": "template_123" }
]
```

3. Items Collection

Stores the actual key-value data using flexible MongoDB BSON documents.

```
{
  "_id": "item_999",
```

```
"categoryId": "cat_petrol",
"data": {
  "brand": "Toyota",
  "year": 2022,
  "price": 25000
}
```

Notice how data is just a dynamic object. MongoDB handles arbitrary key-value pairs naturally inside a single document, which is why your supervisor specifically suggested MongoDB for this!

The Role-Based Access Control (RBAC) Breakdown

Your supervisor requested three clear access tiers:

Role	What They Can Do	Typical User
Template Admin	Can create/edit Templates and Categories. They define the structure of the system.	System Owner / Architect
Data Contributor	Cannot create templates, but can add/edit Items inside existing categories.	Content Managers / Data Entry
Reader	Read-only access. Can browse categories and view item key-value data.	End-users / Public Visitors

Summary Checklist

- **The Goal:** Build a generic dynamic catalog framework, not a static vehicle website.
- **Database Strategy:** Use generic MongoDB collections (Templates, Categories, Items) rather than collection-per-entity.
- **Frontend Strategy:** Render dynamic forms based on the fields array defined in the template.

Would you like to draft a quick set of technical questions or schema definitions together to confirm the exact scope with your supervisor?

User prompt: Can you create a prompt to build this in AntiGravity using gemini 3 pro?

Response: Here is a comprehensive, production-ready prompt tailored for **AntiGravity IDE** using **Gemini 3 Pro**.

It is structured specifically for agentic execution—giving the LLM a clear architecture, database schema, RBAC rules, UI expectations, and a step-by-step rollout plan so it can build the system autonomously without hallucinating unnecessary collections.

Copy & Paste this Prompt into AntiGravity

Act as a Senior Full-Stack Engineer and Architect. We are building a Dynamic Catalog Engine (a metadata-driven mini-CMS)

```
### System Overview
This is a flexible, blank-slate catalog builder where users define data structures dynamically. Rather than creating sta
---

### 1. Database Schema Specifications (MongoDB / Mongoose)

1. **User Schema (`User`)**
  - `name`: String
  - `email`: String (unique)
  - `password`: String (hashed)
  - `role`: Enum [`TEMPLATE_ADMIN`, `DATA_CONTRIBUTOR`, `VIEWER`]

2. **Template Schema (`Template`)**
  - `name`: String (e.g., "Vehicles", "Electronics")
  - `description`: String
  - `fields`: Array of objects:
    - `key`: String (e.g., "engineCapacity")
    - `label`: String (e.g., "Engine Capacity (CC)")
    - `type`: Enum [`string`, `number`, `boolean`, `date`, `select`]
    - `options`: Array of Strings (for select type)
    - `required`: Boolean

3. **Category Schema (`Category`)**
  - `name`: String (e.g., "Cars", "Petrol", "Sedan")
  - `parentId`: ObjectId (ref: `Category`, nullable for top-level categories)
  - `templateId`: ObjectId (ref: `Template`, required)
```

4. ****Item Schema (`Item`)****
 - `categoryId`: ObjectId (ref: `Category`, required)
 - `data`: Record<string, any> (Dynamic key-value pairs validated against the Category's Template fields)

2. Security & RBAC Middleware Requirements

Implement JWT Authentication with middleware to enforce strict Role-Based Access Control:

- ****`TEMPLATE\_ADMIN`****: Full access (Create/Edit/Delete Templates, Categories, and Items).
- ****`DATA\_CONTRIBUTOR`****: Can only Create/Edit/Delete `Items`. Read-only for Templates and Categories.
- ****`VIEWER`****: Read-only access to Categories and `Items`. Blocked from creating or modifying anything.

3. Key Features & Frontend Workflow (React + Tailwind)

1. ****Role Context & Switcher / Auth Flow****:
 - Authentication system with role-based UI rendering (hide action buttons if user lacks permissions).
2. ****Template Builder (Admin Only)****:
 - Interactive UI to create a template and dynamically add/remove custom fields with field types.
3. ****Hierarchical Category Tree View****:
 - Visual breadcrumb / folder view showing Category -> Subcategory relations and assigned Template.
4. ****Dynamic Form Renderer (Contributor / Admin)****:
 - When adding an Item under a Category, fetch its assigned Template and dynamically generate form inputs based on the
5. ****Catalog Viewer (All Roles)****:
 - Table or Grid view rendering dynamic key-value item data under selected categories.

Execution Instructions for AntiGravity Agent

Please execute this build in step-by-step phases, creating clean, modular code:

1. ****Phase 1: Project Setup & Backend Foundation****
 - Setup project structure (`/server` and `/client`).
 - Configure Mongoose connection, models (`User`, `Template`, `Category`, `Item`).
 - Set up Auth Controller & RBAC Middlewares.
2. ****Phase 2: REST API Endpoints****
 - `/api/auth` (Register, Login, Me)
 - `/api/templates` (CRUD - Protected: `TEMPLATE\_ADMIN`)
 - `/api/categories` (CRUD with hierarchical lookup - Protected)
 - `/api/items` (CRUD with dynamic payload validation - Protected: `DATA\_CONTRIBUTOR`, `TEMPLATE\_ADMIN`)
3. ****Phase 3: Frontend Implementation****
 - Scaffolding with Vite + React + Tailwind + Lucide Icons.
 - Auth Context & Route Guards.
 - Template Builder Component.
 - Category Hierarchy Component.
 - Dynamic Form Renderer Component for Items.
 - Item Data Table View.
4. ****Phase 4: Database Seeding****
 - Create a seed script `seed.js` pre-populating:
 - 3 default users (one for each role).
 - 1 Template ("Vehicles").
 - Nested Categories ("Vehicles" -> "Cars" -> "Petrol").
 - 2 sample Items under "Petrol" with dynamic key-value pairs.

Let's start by scaffolding the project directory and writing the backend models and server setup.

Tips for execution in AntiGravity:

- **Run in Agent Mode**: Ensure AntiGravity is in full agent mode so it can execute `npm init`, set up files in bulk, and run the backend server or seed scripts directly.
- **Database Note**: If you don't have a local MongoDB running, ask the agent to use `mongodb-memory-server` in the node backend so it runs out-of-the-box without requiring an external database link during development.

User prompt: What do you call systems like these? And can you suggest a name for it ?

Response: In software development, systems like this fall under a few well-defined architectural terms depending on which angle you are looking at it from:

Technical Names for This System

- **Headless CMS (Content Management System):** This is the most accurate industry term. Systems like *Strapi*, *Sanity*, or *Directus* work exactly like this. They provide an admin UI to define schema fields (templates) and store dynamic content, which is then served via an API.
- **Metadata-Driven Architecture (or Dynamic Schema System):** In developer terms, this describes *how* it's built. Instead of hardcoding database schemas, the structure of the data is defined by "metadata" (your Templates collection).
- **EAV (Entity-Attribute-Value) Model:** A classic design pattern where entities (Items) have dynamic attributes (Keys) and values without needing separate tables for every category.
- **Dynamic Data Engine / Dynamic Catalog Builder:** A functional term commonly used in enterprise apps when creating flexible product catalogs or inventory tools.

Software Name Ideas

Depending on the "vibe" you want to present to your supervisor, here are a few naming suggestions broken down by style:

1. Developer / Tech-Forward Names

- **SchemaShift** (*Highlights that the data schema can shift and adapt on the fly*)
- **MetaForge** (*Built on the concept of forging templates and metadata*)
- **FlexiBase** (*Flexible, dynamic base architecture*)
- **NEXUS Catalog** (*A central hub connecting templates, categories, and items*)

2. Modern & Minimalist (SaaS Style)

- **OmniCatalog** (*One catalog engine for literally anything*)
- **Dynamicus** (*Clean, catchy, sounds like a framework*)
- **Kibo** (*Short, punchy—derived from "Key-Item Builder Object"*)
- **AdaptiForm** (*Focuses on forms and fields that adapt to any template*)

3. Practical / Enterprise Names

- **Dynamic Entity Engine (DEE)**
- **Universal Template Framework (UTF)**
- **MetaCatalog System**

Recommendation: Calling it something like **"OmniCatalog"** or **"SchemaShift"** on your presentation or pull requests will immediately make the project sound well-engineered and structured!

User prompt: Which stack do you recommend for this? I would like to create it like the supabase or resend website in terms of UI/UX. Would it be great if I use typescript and nextjs with vercel? As we are scaling this, should we use mongodb or something else for the database tech? It should be rigid and fast too.

Response: Yes, **TypeScript**, **Next.js (App Router)**, and **Vercel** are the ideal stack for achieving the polished, high-end feel of platforms like Supabase or Resend.

For the database layer, while your supervisor mentioned MongoDB, evaluating PostgreSQL—specifically through **PostgreSQL + JSONB** on **Supabase**—reveals key performance and scaling advantages for this specific architecture.

The Recommended Stack

Layer	Technology	Why It Fits
Framework	Next.js (App Router)	Instant page loads, Server Components, built-in Server Actions, and easy Vercel deployment.
Language	TypeScript	Crucial for dynamic key-value forms. Prevents runtime undefined errors when parsing custom user inputs.
Styling & UI	Tailwind CSS + shadcn/ui	The exact design system engine behind Supabase and Resend's dark-mode, minimalist aesthetics.

Layer	Technology	Why It Fits
Animation	Framer Motion	Delivers sleek, responsive transitions when adding form fields or expanding category trees.
Database	PostgreSQL (via Supabase)	High speed, built-in dynamic JSON query support (JSONB), Row-Level Security, and instant Auth.

Database Architecture: Supabase (PostgreSQL) vs. MongoDB

If you need this platform to be **rigid, fast, and scalable**, **PostgreSQL (via Supabase)** outperforms traditional NoSQL for this specific metadata-driven use case.

1. The PostgreSQL Advantage: Native JSONB

PostgreSQL provides a native data type called JSONB (Binary JSON). It allows you to store dynamic key-value pairs while giving you access to indexed, lightning-fast queries and strict relational integrity across categories and templates.

```
-- PostgreSQL / Supabase schema for dynamic items
CREATE TABLE items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id UUID REFERENCES categories(id) ON DELETE CASCADE,
  -- Dynamic key-value data stored natively with GIN indexing
  data JSONB NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Querying nested JSON directly in Postgres:
SELECT * FROM items WHERE data->>'brand' = 'Toyota';
```

2. Direct Feature Comparison

DATABASE COMPARISON	
PostgreSQL (Supabase)	MongoDB
Rigid relations + JSONB Built-in Row-Level Security Ultra-fast indexed queries Integrated Auth & Edge APIs	Flexible document storage Application-level auth Requires horizontal sharding Manual Auth / Driver setup

- **Relational Rigidity + Schemaless Speed:** Categories, templates, and permissions require strict integrity (relational). The custom field values need flexible storage (JSONB). Postgres handles both inside a single engine.
- **Built-in RBAC via Row-Level Security (RLS):** Supabase allows you to write security policies directly at the database layer, keeping the backend secure out of the box.

Achieving the "Supabase / Resend" Look & Feel

To match the clean, developer-focused aesthetic of Supabase or Resend:

1. **Use shadcn/ui Components:** Set up a dark theme using zinc or slate palettes, compact borders, subtle hover effects, and clean monochrome styling.
2. **Command Palette (cmd + k):** Include a search modal using cmdk so users can rapidly jump between categories or templates.
3. **Optimistic UI Updates:** Use React useOptimistic hooks so UI changes (like adding a field or category) render instantly before the network call completes.

Summary Recommendation

- **Use Next.js + TypeScript + Vercel.** It provides full-stack capabilities with minimal deployment friction.
- **Use Supabase (Postgres) over MongoDB.** Storing dynamic attributes in a Postgres JSONB column gives you dynamic fields alongside strict foreign-key relations, built-in Auth, and speed. *(If your supervisor strictly mandates MongoDB, the same logic applies—store dynamic attributes inside an unstructured sub-document).*

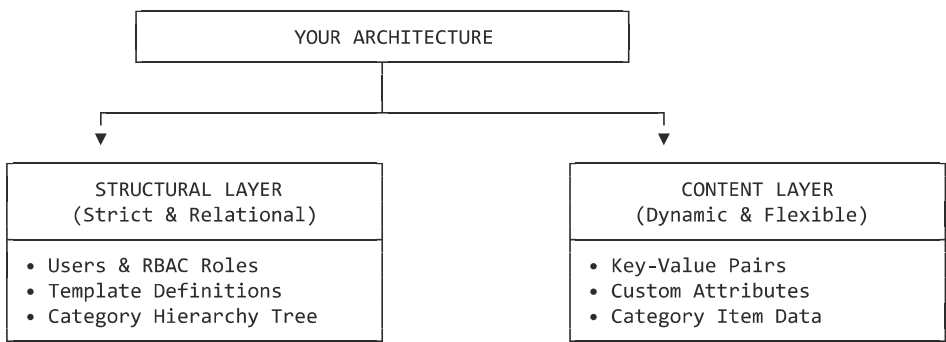
User prompt: Can you elaborate more on Supabase vs MongoDB? What are the pros and cons? Which one wins ultimately for this project specifically?

Response: Here is the deep dive into **Supabase (PostgreSQL) vs. MongoDB** for your dynamic catalog project, breaking down how each handles dynamic data, where they excel, and which one ultimately comes out on top.

The Core Difference: How They Handle This Project

To understand why one fits better, look at what your app is actually doing. Your app has two distinct layers:

- 1. **The Structural Layer (Strict):** Users, Roles (RBAC), Categories, Subcategories (Parent-Child Tree), and Templates. This data is **highly relational**.
- 2. **The Content Layer (Dynamic):** The actual key-value attributes inside an Item (e.g., {"engine": "V8", "doors": 2}). This data is **schemaless**.



Here is how each database tackles these two layers:

- **MongoDB** treats *everything* as dynamic documents.
- **Supabase (PostgreSQL)** gives you a **hybrid engine**: strict relational tables for your structure + native JSONB columns for your dynamic key-value content.

Pros & Cons Breakdown

Option 1: Supabase (PostgreSQL + JSONB + RLS)

Pros:

- **Best of Both Worlds:** Handles strict categories and roles using Relational Tables, while storing dynamic item data in binary JSON (JSONB).
- **Built-in Security (Row Level Security):** You can define role permissions directly at the database layer. Even if someone bypasses your API, the database will refuse to return or modify data unless the JWT user has the correct role (TEMPLATE_ADMIN, DATA_CONTRIBUTOR, or VIEWER).
- **GIN Indexing on Dynamic Fields:** PostgreSQL lets you create Generalized Inverted Indexes (GIN) on the JSONB column. This means searching inside dynamic key-value pairs (e.g., WHERE data->>'color' = 'Red') remains fast even with millions of rows.
- **Serverless Native:** Integrates smoothly with Next.js, Vercel, and Server Actions via connection poolers without exhausting database connections.
- **Auto-generated TypeScript Types:** Supabase CLI generates strict TypeScript types directly from your database schema, making frontend integration clean and type-safe.

Cons:

- **JSONB Query Syntax:** Querying deeply nested JSON fields in SQL uses specific operators (->, ->>) which take a short time to get used to compared to plain JS objects.

Option 2: MongoDB (Mongoose / Node.js)

Pros:

- **Native Document Structure:** Key-value pairs are stored natively as BSON. Adding or reading arbitrary dynamic fields feels completely natural in JavaScript.
- **Supervisor Alignment:** Your supervisor specifically mentioned MongoDB, so choosing it requires zero justification or convincing on your end.
- **Schema-less Freedom:** No need to define table columns beforehand for the items collection.

Cons:

- **Weak Relational Integrity:** Managing nested category trees (Vehicles -> Cars -> Petrol) requires manual handling in application code. If a parent category is deleted, MongoDB won't automatically clean up or protect orphaned subcategories unless you write complex hooks.

- **Application-Only Security:** MongoDB has no built-in Row Level Security for end-users. You must write, test, and maintain custom Express/Next.js middleware to enforce your 3 roles (TEMPLATE_ADMIN, DATA_CONTRIBUTOR, VIEWER) across every endpoint.
- **Serverless Connection Pitfalls:** Connecting MongoDB to Next.js on Vercel requires careful connection caching (global._mongooseClient) to avoid opening a new database connection on every serverless function call.

Feature Comparison Table

Feature / Metric	Supabase (Postgres)	MongoDB
Category Tree Hierarchy	Winner (Foreign keys & recursive queries)	Harder (Requires manual application logic)
Dynamic Key-Value Storage	Equal (JSONB column)	Equal (Native BSON document)
Role-Based Security (RBAC)	Winner (Database-level Row Level Security)	Harder (Application-level middleware only)
Performance at Scale	Winner (Strict indexing + GIN indexes)	Good (Requires wildcard indexes for sub-fields)
Next.js & Vercel Developer Experience	Winner (Instant setup + type generation)	Moderate (Connection pooling management needed)

The Ultimate Winner for This Project: Supabase

Supabase (PostgreSQL) is the clear structural and architectural winner for this specific build.

Why?

Your supervisor described a system that requires **strict rules on top of dynamic content**:

1. It needs strict permissions (RBAC).
2. It needs strict structural relationships (Templates → Categories → Subcategories → Items).
3. It needs dynamic key-value inputs inside those items.

MongoDB is great when *everything* is unstructured. But because 70% of this app's complexity comes from roles, relationships, and parent-child categories, **PostgreSQL's combination of relational tables + JSONB columns solves the entire puzzle out of the box.**

How to Pitch This to Your Supervisor

If your supervisor asks why you chose Supabase/Postgres over MongoDB, here is the exact architectural response to give:

"MongoDB is great for pure document storage, but our system relies heavily on strict category hierarchies, template references, and role-based access control. I chose PostgreSQL via Supabase because it gives us the best of both worlds: strict relational integrity for our categories and security rules, combined with a native JSONB column that allows dynamic key-value storage exactly like MongoDB. It gives us the flexibility you asked for without sacrificing database speed or data safety."

Would you like me to generate the exact SQL schema (including the JSONB item setup and Row Level Security policies) for Supabase so you can visualize how simple it is?

User prompt: Can you give an updated prompt to AntiGravity again with everything we discussed so far.

Response: Here is the complete, master prompt updated with all technical decisions: **Next.js (App Router), TypeScript, Supabase (PostgreSQL + JSONB + RLS), Tailwind CSS, shadcn/ui, and a Supabase/Resend dark-mode design system.**

Copy and paste this directly into **AntiGravity** for Gemini 3 Pro to execute autonomously.

Copy & Paste this Prompt into AntiGravity

Act as a Principal Full-Stack Engineer and Software Architect. We are building a high-performance, metadata-driven Dynam
The design and UI/UX must mirror the dark-mode, minimalist, developer-first aesthetic of platforms like Supabase and Res

Tech Stack & Architecture
- **Framework:** Next.js (App Router) with TypeScript
- **Database & Auth:** Supabase (PostgreSQL with native `JSONB` storage + Supabase Auth)
- **Styling & Components:** Tailwind CSS, shadcn/ui, Lucide React Icons, Framer Motion
- **State & Data Fetching:** React Server Components, Server Actions, and `@supabase/ssr`

1. Database Schema & Supabase Setup (`schema.sql`)

Provide a SQL migration script for Supabase defining 4 core tables and Row Level Security (RLS) policies:

1. `profiles`
 - `id` (UUID, primary key, references `auth.users.id` ON DELETE CASCADE)
 - `email` (TEXT)
 - `role` (TEXT, check constraint: `role` IN ('TEMPLATE_ADMIN', 'DATA_CONTRIBUTOR', 'VIEWER'), default 'VIEWER')
 - `created\_at` (TIMESTAMPTZ, default `now()`)
2. `templates`
 - `id` (UUID, primary key, default `gen\_random\_uuid()`)
 - `name` (TEXT, e.g., "Vehicles", "Electronics")
 - `description` (TEXT)
 - `fields` (JSONB, array of field objects: `[ { key, label, type, options, required } ]`)
 - `type` options: `string` | `number` | `boolean` | `date` | `select`
 - `created\_at` (TIMESTAMPTZ, default `now()`)
3. `categories`
 - `id` (UUID, primary key, default `gen\_random\_uuid()`)
 - `name` (TEXT, e.g., "Cars", "Petrol", "Electric")
 - `parent\_id` (UUID, nullable, references `categories(id)` ON DELETE CASCADE)
 - `template\_id` (UUID, references `templates(id)` ON DELETE RESTRICT)
 - `created\_at` (TIMESTAMPTZ, default `now()`)
4. `items`
 - `id` (UUID, primary key, default `gen\_random\_uuid()`)
 - `category\_id` (UUID, references `categories(id)` ON DELETE CASCADE)
 - `data` (JSONB, dynamic key-value pairs matching the assigned template fields)
 - `created\_at` (TIMESTAMPTZ, default `now()`)
5. ****Indexes & RLS Policies:****
 - Create a GIN index on `items` USING gin (data) for fast dynamic JSON queries.
 - Configure Supabase RLS Policies:
 - `TEMPLATE\_ADMIN`: Full access (SELECT, INSERT, UPDATE, DELETE) across all tables.
 - `DATA\_CONTRIBUTOR`: SELECT on `profiles`, `templates`, `categories`; Full access on `items`.
 - `VIEWER`: SELECT-only on `templates`, `categories`, and `items`.

2. Core Features & UI Layout

1. ****Supabase / Resend Inspired Design System:****
 - Dark theme enabled by default (`bg-zinc-950 text-zinc-100 border-zinc-800`).
 - Command Palette modal (`Cmd + K` / `Ctrl + K`) using `cmdk` to jump between templates, categories, and settings.
 - Clean top navigation bar showing active user role badge and a role switcher modal (for local testing/debugging).
2. ****Template Builder Component (`TEMPLATE\_ADMIN` Only):****
 - Dynamic form builder allowing admins to name a template and interactively add, configure, reorder, or remove custom
3. ****Hierarchical Category Tree Visualizer:****
 - Interactive tree/folder view displaying nested parent-child categories (e.g., `Vehicles -> Cars -> Petrol`).
 - Ability to add subcategories and link them to a specific Template.
4. ****Dynamic Item Form Renderer (`DATA\_CONTRIBUTOR` & `TEMPLATE\_ADMIN`):****
 - When a user selects a category to add an item, fetch the associated `template.fields`.
 - Automatically generate appropriate shadcn/ui input fields based on field types (`string` -> Input, `number` -> Numb
5. ****Data Grid Catalog Viewer:****
 - Dynamic table view rendering `items` under selected categories.
 - Automatically extract JSON keys into table columns.
 - Include search, sorting, and JSON filtering capabilities.

3. Step-by-Step Implementation Instructions for AntiGravity

Execute this project sequentially in modular steps:

- **Phase 1: Environment & Project Setup****
- Initialize Next.js (App Router) with TypeScript, Tailwind CSS, shadcn/ui, and Lucide Icons.
 - Install `@supabase/supabase-js`, `@supabase/ssr`, and `cmdk`.
 - Create `.env.local` template with `NEXT\_PUBLIC\_SUPABASE\_URL` and `NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY`.

****Phase 2: Database Migration & Supabase Client Setup****

- Generate `supabase/schema.sql` containing all table definitions, GIN indexing, trigger functions for profile creation
- Build server-side (`utils/supabase/server.ts`) and client-side (`utils/supabase/client.ts`) Supabase utilities.

****Phase 3: Auth & Dynamic Layout Shell****

- Build Auth routes (`/login`, `/signup`) and middleware for role checks.
- Build main App Dashboard shell with dark-mode sidebar, header with active role indicator, and Command Palette (Cmd +

****Phase 4: Template & Category Builder Features****

- Build `/templates` routes (Template List and Interactive Builder).
- Build `/categories` route with dynamic recursive hierarchy view.

****Phase 5: Dynamic Item CRUD & Data Table****

- Build dynamic form parser component that renders UI controls from a `Template` JSON definition.
- Build `/catalog` route displaying dynamic table grids populated from `items.data`.

****Phase 6: Seed Script & Verification****

- Create `supabase/seed.sql` pre-populating:
 - 3 Test profiles (Admin, Contributor, Viewer).
 - 1 "Vehicles" Template with fields (`brand`, `modelYear`, `isElectric`, `fuelType`).
 - Hierarchical categories (`Vehicles` -> `Cars` -> `Petrol`).
 - 2 sample `items` with dynamic `JSONB` data.

Let's begin by scaffolding the Next.js project directory, configuring Tailwind CSS dark mode with shadcn/ui defaults, an
